

DBMS — COMPLETE INTERVIEW NOTES

Target: College Exams · Placement · Product-Based Company Interviews

UNIT 1 — DBMS FUNDAMENTALS & RELATIONAL MODEL

1. Database Basics

Term	Meaning
Data	Raw facts (e.g., "25", "John")
Information	Processed data with meaning
Database	Organised collection of related data
DBMS	Software to create, manage, query databases
RDBMS	DBMS with relational model (tables + SQL)

DBMS vs File System

DBMS	File System
Avoids redundancy	Data duplication
ACID transactions	No transaction support
Multi-user concurrency	Manual locking
Security & access control	OS-level only

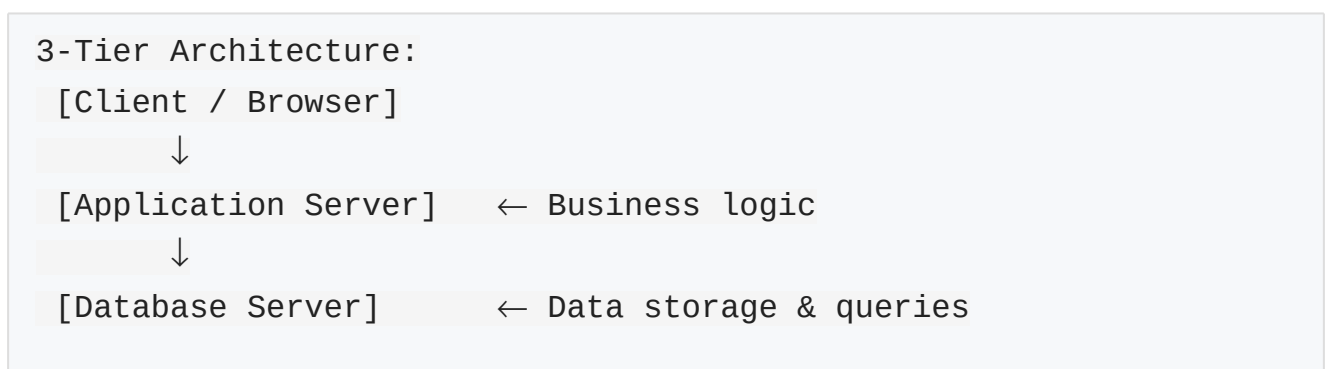
Data independence	App tightly coupled to data
-------------------	-----------------------------

Why DBMS over files? Consistency, sharing, recovery, security, and querying at scale.

Database Users: End users · App developers · DB admins (DBA)

DBA Responsibilities: Schema design, performance tuning, backup/recovery, user access, monitoring.

2. DBMS Architecture



- **1-tier:** App and DB on same machine (desktop apps).
- **2-tier:** Client talks directly to DB (thin clients).
- **3-tier:** Client → App layer → DB (modern web apps). Best separation of concerns.

3. Schema vs Instance

Schema	Instance
Structure/blueprint of DB	Actual data at a point in time
Changes rarely	Changes with every INSERT/UPDATE/DELETE
Like a class definition	Like an object of that class

Example: Schema = `students(id, name, age)` · Instance = the 500 rows currently stored.

4. Data Abstraction & Independence

```
View Level    ← What user sees (customised views)
  ↓
Logical Level ← Tables, columns, relationships
  ↓
Physical Level ← How data stored on disk (files, pages)
```

- **Physical independence:** Change storage structure without affecting logical schema.
 - **Logical independence:** Change logical schema without affecting views/apps.
-

5. Database Models (Brief)

Model	Key Idea
Hierarchical	Tree structure, parent-child
Network	Graph, multiple parents allowed
Relational	Tables, SQL — most widely used
Object-oriented	Data as objects
NoSQL	Document/key-value/graph/column

6. Relational Model

Relation = Table. **Tuple** = Row. **Attribute** = Column. **Domain** = allowed values.

Degree = number of columns. **Cardinality** = number of rows.

```
students(id, name, dept) ← Relation schema
( 1, Alice, CS )         ← Tuple
( 2, Bob, EE )
Degree = 3, Cardinality = 2
```

7. Keys

Super Key → remove attributes → Candidate Key → choose one → Pri

Key	Definition
Super key	Any set of attributes that uniquely identifies a row
Candidate key	Minimal super key (no redundant attributes)
Primary key	Chosen candidate key; NOT NULL, unique
Alternate key	Candidate keys not chosen as PK
Foreign key	Attribute referencing PK of another table
Composite key	Key made of two or more attributes
Natural key	Real-world attribute (e.g., email)
Surrogate key	System-generated ID (e.g., auto-increment id)

Primary Key vs Unique Key: PK cannot be NULL; table has one PK. Unique allows one NULL; multiple unique constraints allowed.

Foreign Key: Enforces referential integrity — value must exist in parent table or be NULL.

Interview Point: A table can have many candidate keys but only one primary key.

8. Constraints

Constraint	Effect
NOT NULL	Column must have a value
UNIQUE	No two rows share the same value
PRIMARY KEY	NOT NULL + UNIQUE
FOREIGN KEY	References PK in parent table
CHECK	Value must satisfy a condition
DEFAULT	Provides value when none given
Entity Integrity	PK cannot be NULL
Referential Integrity	FK must reference an existing PK

Violation: INSERT/UPDATE fails; with FK you can set ON DELETE CASCADE or RESTRICT.

9–10. ER Model & Relationships

Entity = real-world object (Student, Product). **Entity set** = collection of similar entities.

Attribute Types: - Simple: atomic (age) - Composite: has sub-parts (address → city, state) - Single-valued / Multi-valued (phone numbers) - Derived: computed (age from DOB)

Cardinality:

Type	Meaning	Example
1:1	One entity maps to one	Person ↔ Passport
1:N	One to many	Dept → Employees
M:N	Many to many	Students ↔ Courses

Participation: Total (double line = every entity must participate) · Partial (single line = optional).

11. Advanced ER

- **Strong entity:** Has its own PK (e.g., Student).
 - **Weak entity:** No PK; depends on strong entity (e.g., Dependent relies on Employee). Identified by discriminator + owner's PK.
 - **Generalisation:** Combine subclasses → superclass (bottom-up).
 - **Specialisation:** Split superclass → subclasses (top-down).
 - **Aggregation:** Treat a relationship as an entity for another relationship.
-

12. ER to Relational Mapping

ER Concept	Relational Mapping
Strong entity	Table with PK
Attribute	Column
1:1 relationship	Add FK to either side
1:N relationship	Add FK on the N side

M:N relationship	New junction table (FK1, FK2)
Weak entity	Table with owner's PK + discriminator as composite PK
Multi-valued attribute	Separate table (entity PK + attribute value)

Unit 1 Interview Essentials

1. DBMS vs File System — must know all differences
2. 3-tier architecture purpose
3. Schema vs instance analogy (class vs object)
4. Three levels of abstraction + independence
5. Super key → candidate key → primary key chain
6. Primary key vs unique key (NULL behaviour)
7. Foreign key + referential integrity + ON DELETE actions
8. All ER attribute types with examples
9. M:N relationship → junction table mapping
10. Weak entity — what and why
11. Total vs partial participation
12. Surrogate vs natural key trade-offs

Common Interview Traps: - PK and FK can exist on the same column (self-referencing). - A table can have zero foreign keys. - A weak entity can still have attributes; it just cannot have its own PK without the owner. - Generalisation and specialisation are inverses of each other.

UNIT 2 — SQL & RELATIONAL QUERYING

1. SQL Categories

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE	Define structure
DML	INSERT, UPDATE, DELETE	Manipulate data
DQL	SELECT	Query data
DCL	GRANT, REVOKE	Access control
TCL	COMMIT, ROLLBACK, SAVEPOINT	Transaction control

2. DDL Commands & Key Comparison

```
CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50), salary DECIMAL(10,2));
ALTER TABLE employees ADD COLUMN dept_id INT;
DROP TABLE employees;           -- Deletes table + data + structure permanent
TRUNCATE TABLE employees;       -- Deletes all rows; keeps structure; faster
```

Feature	DELETE	TRUNCATE	DROP
Removes rows	Yes (with WHERE)	All rows	Entire table
WHERE clause	Yes	No	No
Rollback	Yes (DML)	No (DDL)	No
Triggers fired	Yes	No	No

Auto-increment reset	No	Yes	N/A
----------------------	----	-----	-----

3–4. DML & SELECT

```
INSERT INTO employees VALUES (1, 'Alice', 80000);
UPDATE employees SET salary = 90000 WHERE id = 1;
DELETE FROM employees WHERE id = 1;
SELECT name, salary FROM employees WHERE dept_id = 3 ORDER BY salary DESC
```

Clauses: `DISTINCT` removes duplicates · `AS` creates alias · `OFFSET n` skips n rows.

5. Operators

Type	Operators
Arithmetic	+, -, *, /, %
Comparison	=, !=, <, >, <=, >=
Logical	AND, OR, NOT
Pattern	LIKE ('%ali%', '_at')
Range	BETWEEN a AND b
Membership	IN (list), NOT IN
Null check	IS NULL, IS NOT NULL

6. Aggregate Functions & NULL

Function	Returns	NULL handling
COUNT(*)	All rows	Counts NULLs
COUNT(col)	Non-NULL rows	Ignores NULLs
SUM, AVG, MIN, MAX	Result	Ignores NULLs

$AVG(salary) = SUM / \text{count of non-NULL salary rows, not total rows.}$

7. GROUP BY, HAVING & Logical Query Order

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

- **WHERE** filters individual rows (before grouping).
- **HAVING** filters groups (after GROUP BY).

```
SELECT dept_id, AVG(salary) FROM employees
WHERE active = 1
GROUP BY dept_id
HAVING AVG(salary) > 50000
ORDER BY AVG(salary) DESC;
```

Interview Point: You cannot use aggregate functions in WHERE; use HAVING.

8. Joins

Join	Returns
INNER JOIN	Matching rows in both tables

LEFT JOIN	All left rows + matched right (NULL if no match)
RIGHT JOIN	All right rows + matched left (NULL if no match)
FULL OUTER JOIN	All rows from both; NULL where no match
CROSS JOIN	Cartesian product (every combination)
SELF JOIN	Table joined with itself

```
-- Employees with no department (LEFT JOIN pattern)
SELECT e.name FROM employees e LEFT JOIN departments d ON e.dept_id = d.id
WHERE d.id IS NULL;
```

Interview Point: JOIN vs Subquery — JOINS are usually faster (optimizer handles well); correlated subqueries can be slower.

9. Subqueries

- **Single-row:** returns one value → use `=`
- **Multi-row:** returns multiple values → use `IN`, `ANY`, `ALL`
- **Correlated subquery:** references outer query; re-executes for each outer row.
- **EXISTS:** returns TRUE if subquery returns any rows (stops at first match — efficient).

```
-- Correlated: employees earning above department average
SELECT name FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e1.dept_id);
```

10. Set Operations

Operation	Result
UNION	Distinct rows from both queries
UNION ALL	All rows including duplicates (faster)
INTERSECT	Rows in both queries
EXCEPT/MINUS	Rows in first but not second

Requirement: Same number of columns, compatible data types.

11. NULL — Three-Valued Logic

NULL = unknown/missing. $\neq 0$, $\neq$ empty string.

`NULL = NULL` $\rightarrow$ UNKNOWN (use `IS NULL` instead).

A	B	A AND B	A OR B
T	U	UNKNOWN	TRUE
F	U	FALSE	UNKNOWN
U	U	UNKNOWN	UNKNOWN

```
COALESCE(salary, 0)  -- returns first non-NULL value
NULLIF(a, b)         -- returns NULL if a = b
```

12. SQL Functions (Key Ones)

```
-- String
UPPER, LOWER, TRIM, LENGTH, SUBSTRING(str,1,3), CONCAT, REPLACE

-- Numeric
ROUND(3.14159, 2), CEIL(4.1)=5, FLOOR(4.9)=4, ABS(-5)=5, MOD(10,3)=1

-- Date
NOW(), CURDATE(), DATEDIFF(d1,d2), DATE_ADD(d, INTERVAL 1 MONTH)

-- Conditional
CASE WHEN salary > 50000 THEN 'High' ELSE 'Low' END
COALESCE(col1, col2, 'default')
```

13. Views & Materialized Views

View: Virtual table based on a SELECT query. Data not stored.

```
CREATE VIEW high_earners AS SELECT name, salary FROM employees WHERE salary > 50000
```

- Simplifies complex queries · Provides security (hide columns) · Does not store data.
- **Updatable views:** Simple views on single table (no aggregates/joins) can INSERT/UPDATE.
- **Materialized view:** Result stored on disk · Faster reads · Must be refreshed.

14. CTE (Common Table Expression)

```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) avg_sal FROM employees GROUP BY dept_id
)
SELECT e.name FROM employees e JOIN dept_avg d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_sal;
```

- **CTE vs Subquery:** CTE is named, reusable in same query, more readable.

- **Recursive CTE:** Query references itself → used for hierarchies (org charts, trees).

```
WITH RECURSIVE tree AS (  
  SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL  
  UNION ALL  
  SELECT e.id, e.name, e.manager_id FROM employees e JOIN tree t ON e.manag  
)  
SELECT * FROM tree;
```

15. Window Functions ■

```
SELECT name, dept_id, salary,  
  ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn,  
  RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,  
  DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS drnk,  
  LAG(salary) OVER (PARTITION BY dept_id ORDER BY salary) AS prev_sal,  
  LEAD(salary) OVER (PARTITION BY dept_id ORDER BY salary) AS next_sal,  
  SUM(salary) OVER (PARTITION BY dept_id ORDER BY salary ROWS UNBOUNDED PR  
FROM employees;
```

Function	Ties	Gap in rank?
ROW_NUMBER	Arbitrary order	No ties
RANK	Same rank	Yes (1,1,3)
DENSE_RANK	Same rank	No gap (1,1,2)

16. Important SQL Interview Problems

```
-- 2nd highest salary  
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM
```

```

-- or with DENSE_RANK
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC

-- Nth highest salary (N=3)
SELECT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 2;

-- Top salary per department
SELECT dept_id, MAX(salary) FROM employees GROUP BY dept_id;

-- Top 3 per department (window function)
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) d
) t WHERE dr <= 3;

-- Employees earning more than their manager
SELECT e.name FROM employees e JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;

-- Duplicate rows
SELECT email, COUNT(*) FROM employees GROUP BY email HAVING COUNT(*) > 1;

-- Delete duplicates (keep min id)
DELETE FROM employees WHERE id NOT IN (SELECT MIN(id) FROM employees GROUP

-- Customers with no orders
SELECT c.name FROM customers c LEFT JOIN orders o ON c.id = o.customer_id

-- Running total
SELECT date, amount, SUM(amount) OVER (ORDER BY date) AS running_total FROM

-- YoY growth
SELECT year, revenue,
  LAG(revenue) OVER (ORDER BY year) AS prev,
  ROUND(100.0*(revenue - LAG(revenue) OVER (ORDER BY year)) / LAG(revenue)
FROM annual_sales;

```

SQL Interview Checklist

- Write any JOIN from memory · Know all aggregate NULL behaviour · Use window functions confidently
- Know RANK vs DENSE_RANK vs ROW_NUMBER · Write correlated subqueries · Write recursive CTE
- Know logical query order · WHERE vs HAVING · DELETE vs TRUNCATE vs DROP
- Solve: 2nd highest salary · top-N per group · employees > manager · duplicates

Common Interview Traps: - `COUNT(col) ≠ COUNT(*)` when NULLs exist. - Window functions do NOT reduce row count (unlike GROUP BY). - HAVING can reference SELECT aliases in some DBs but not all. - UNION sorts + deduplicates; always slower than UNION ALL.

UNIT 3 — DATABASE DESIGN, DEPENDENCIES & NORMALIZATION

1. Functional Dependency (FD)

$A \rightarrow B$: "A determines B" — knowing A's value tells you B's value.

- **Determinant:** left side (A) · **Dependent:** right side (B)
- **Trivial FD:** $B \subseteq A$ (e.g., $\{A,B\} \rightarrow A$) — always holds.
- **Non-trivial FD:** $B \not\subseteq A$.
- **Partial dependency:** Part of a composite key determines a non-key attribute.
- **Transitive dependency:** $A \rightarrow B \rightarrow C$ (C depends on non-key B).

Example: `StudentID` $\rightarrow$ `Name` means each StudentID has exactly one Name.

2. Armstrong's Axioms

Axiom	Rule
Reflexivity	If $B \subseteq A$, then $A \rightarrow B$

Augmentation	If $A \rightarrow B$, then $AC \rightarrow BC$
Transitivity	If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$
Union	If $A \rightarrow B$ and $A \rightarrow C$ then $A \rightarrow BC$
Decomposition	If $A \rightarrow BC$ then $A \rightarrow B$ and $A \rightarrow C$

3. Attribute Closure (A^+)

A^+ = all attributes that A can determine using given FDs.

Use: Find candidate keys, check if a key, verify FD membership.

FDs: $A \rightarrow B$, $B \rightarrow C$, $A \rightarrow D$

Find A^+ :

Start: $\{A\}$

$A \rightarrow B \rightarrow \{A, B\}$

$B \rightarrow C \rightarrow \{A, B, C\}$

$A \rightarrow D \rightarrow \{A, B, C, D\}$

$A^+ = \{A, B, C, D\}$

If $A^+ = \text{all attributes} \rightarrow A$ is a super key. Check minimality $\rightarrow$ candidate key.

4. Normalization — Why & How

Goals: Eliminate redundancy · Prevent anomalies · Ensure consistency.

Unnormalized $\rightarrow$ 1NF $\rightarrow$ 2NF $\rightarrow$ 3NF $\rightarrow$ BCNF

Anomalies: - **Insertion:** Can't insert data without other data being present. - **Update:** Changing one fact requires multiple row updates. - **Deletion:** Deleting a row loses other facts.

5–9. Normal Forms

NF	Condition	Removes
1NF	Atomic values; no repeating groups	Multi-valued/composite cells
2NF	1NF + no partial dependency (non-key depends on entire PK)	Partial dependency
3NF	2NF + no transitive dependency (non-key depends on non-key)	Transitive dependency
BCNF	Every determinant is a candidate key	Anomalies 3NF misses

3NF vs BCNF: 3NF allows $A \rightarrow B$ where A is not a candidate key IF B is part of a candidate key. BCNF is stricter: every determinant must be a candidate key. BCNF may lose dependency preservation.

10. Normalization Example (Compact)

```
ORDER(OID, CID, CName, PID, PName, Qty)
```

```
FDs: OID, PID  $\rightarrow$  Qty      (full dependency on composite PK)
```

```
      CID  $\rightarrow$  CName        (partial – CID is part of PK)
```

```
      PID  $\rightarrow$  PName        (partial – PID is part of PK)
```

```
1NF: Already atomic.
```

```
2NF: Remove partial deps:
```

```
      ORDER(OID, CID, PID, Qty)
```

```
      CUSTOMER(CID, CName)
```

```
PRODUCT(PID, PName)
3NF/BCNF: No transitive deps remain → done.
```

11. Lossless vs Dependency Preservation

Property	Meaning
Lossless decomposition	Natural join of decomposed tables = original table (no spurious tuples)
Dependency preservation	All FDs can be checked within individual tables

BCNF guarantees lossless decomposition but may not preserve dependencies. 3NF guarantees both lossless and dependency-preserving decomposition.

12. Denormalization

Why? Normalization increases joins — for read-heavy/reporting systems, pre-joining data into fewer tables improves query speed.

Trade-off: More storage + update anomaly risk vs faster reads.

Interview: "Why not normalize everything?" — In OLAP/analytics, joins are expensive. Denormalized tables (star/snowflake schemas) are common in data warehouses.

13. Database Design Case Studies

E-commerce: User(uid) · Product(pid, cid) · Category(cid) ·
Cart(cart_id, uid) · CartItem(cart_id, pid, qty) · Order(oid, uid,
payment_id) · OrderItem(oid, pid, qty, price) · Payment(payment_id)
· Review(rid, uid, pid, rating)

College: Student(sid) · Department(did) · Course(cid, did) ·
Faculty(fid, did) · Enrollment(sid, cid, grade)

Banking: Customer(cid) · Account(acc_id, cid, branch_id) · Transaction(txn_id, acc_id) · Branch(branch_id) · Loan(loan_id, cid)

Social Media: User(uid) · Post(pid, uid) · Comment(cid, pid, uid) · Like(uid, pid) · Follow(follower_uid, followee_uid)

Normalization Interview Checklist

- Know FD definition + examples · Compute attribute closure step-by-step
- State conditions for 1NF/2NF/3NF/BCNF precisely
- Identify partial and transitive dependencies from a schema
- Explain 3NF vs BCNF difference with example
- Know lossless join vs dependency preservation trade-off
- Explain denormalization and when to use it

Common Interview Traps: - 2NF is only an issue when the candidate key is composite. - A relation in BCNF is always in 3NF, not vice versa. - Normalizing too much can hurt performance (too many joins). - BCNF decomposition can lose dependency preservation.

UNIT 4 — TRANSACTIONS, CONCURRENCY & RECOVERY

1. Transaction & ACID

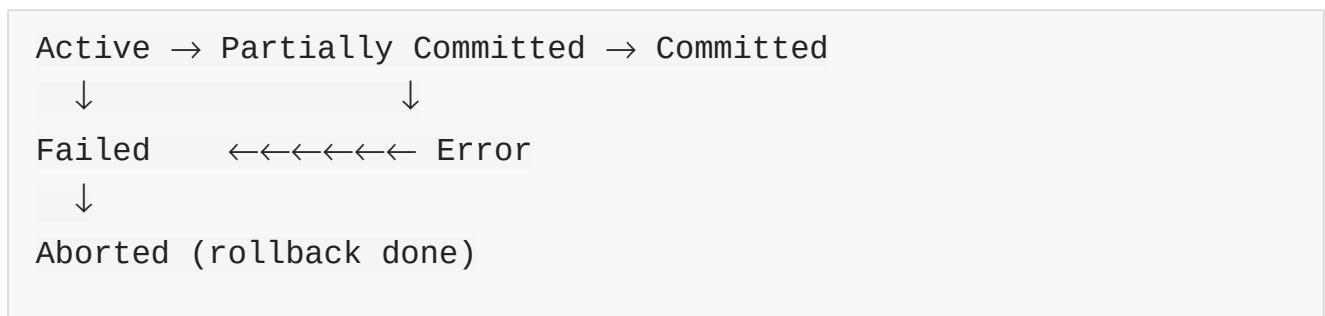
Transaction: A logical unit of work — either all operations succeed or none do.

Banking transfer: $A \rightarrow B$
DEBIT A (UPDATE accounts SET bal = bal-500 WHERE id=A)
CREDIT B (UPDATE accounts SET bal = bal+500 WHERE id=B)
Must both succeed or both roll back.

ACID	Meaning	Example
Atomicity	All or nothing	Transfer: both debit+credit happen or neither

Consistency	DB moves from valid state to valid state	Balance never goes negative (CHECK constraint)
Isolation	Concurrent txns appear sequential	T1's uncommitted debit invisible to T2
Durability	Committed data survives crashes	Disk write (WAL) ensures commit persists

2. Transaction States



3. Schedules & Serializability

- **Serial schedule:** Transactions run one after another — always correct.
- **Concurrent schedule:** Operations interleaved — may or may not be correct.
- **Serializable schedule:** Equivalent to some serial schedule — correct.

Conflict Serializability: Two operations conflict if: different transactions + same data item + at least one is WRITE.

Ops	Conflict?
R1(X), R2(X)	No
R1(X), W2(X)	Yes

W1(X), W2(X)	Yes
--------------	-----

Precedence Graph: Node per transaction. Draw edge $T_i \rightarrow T_j$ when T_i 's operation conflicts and appears before T_j 's. - **No cycle** → **Conflict Serializable** ✓ - **Cycle** → **Not conflict serializable** ✗

T1: R(X), W(Y) T2: R(Y), W(X) Edge $T_1 \rightarrow T_2$ (R1X before W2X) and $T_2 \rightarrow T_1$ (R2Y before W1Y) → CYCLE → not serializable
--

4. Concurrency Problems

Problem	What happens	Prevented by isolation level
Lost Update	T2 overwrites T1's uncommitted write	Repeatable Read
Dirty Read	T2 reads T1's uncommitted change; T1 rolls back	Read Committed
Non-repeatable Read	T2 reads same row twice; T1 commits update in between	Repeatable Read
Phantom Read	T2 re-runs query; T1 inserted new matching rows in between	Serializable

5. Locks & 2PL

Shared Lock (S): Read lock. **Exclusive Lock (X):** Write lock.

	S	X
	S ✓	X
	X X	X

Two-Phase Locking (2PL):

Growing Phase: acquire locks (no releasing)

Lock Point

Shrinking Phase: release locks (no acquiring)

- **Strict 2PL:** Hold all exclusive locks until COMMIT/ABORT → prevents dirty reads, cascading rollbacks.

Why strict 2PL? Ensures recoverability and avoids cascading aborts.

6. Deadlock

T1 holds A, waits for B

T2 holds B, waits for A → Circular wait → Deadlock

4 Necessary Conditions: Mutual exclusion · Hold & wait · No preemption · Circular wait.

Handling: - **Detection:** Wait-for graph — cycle = deadlock; victim chosen and aborted. -

Prevention: Remove one of 4 conditions (e.g., acquire all locks at start). - **Avoidance:**

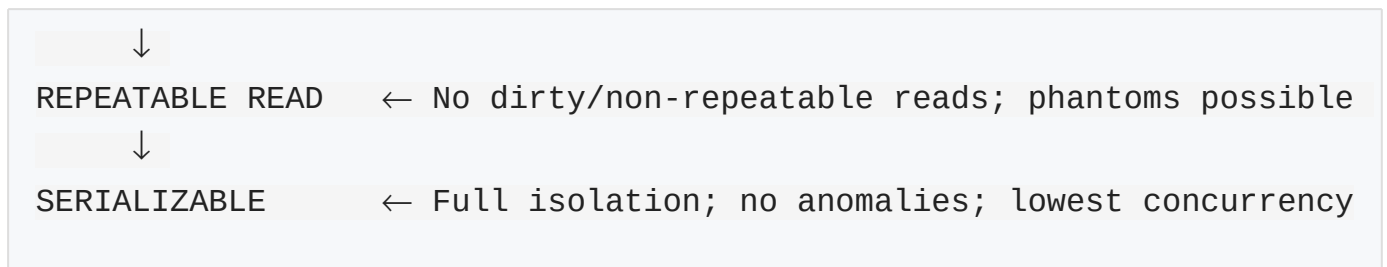
Banker's algorithm — grant locks only if safe state reached. - **Timeout:** Abort transaction if waiting too long.

7. Isolation Levels

READ UNCOMMITTED ← Dirty reads possible

↓

READ COMMITTED ← No dirty reads; non-repeatable reads possible



Level	Dirty	Non-repeatable	Phantom
READ UNCOMMITTED	✓	✓	✓
READ COMMITTED	✗	✓	✓
REPEATABLE READ	✗	✗	✓
SERIALIZABLE	✗	✗	✗

Interview Point: Higher isolation = fewer anomalies but lower concurrency and throughput.

8. MVCC

Multi-Version Concurrency Control: Each transaction sees a snapshot of the DB at its start time. Writers create new versions; readers see old versions → readers don't block writers and vice versa.

Used by: PostgreSQL, MySQL InnoDB, Oracle. Avoids read-write lock contention.

Trade-off: Old versions must be garbage collected (VACUUM in PostgreSQL).

9. Recovery — WAL, Logging, Undo/Redo

Write-Ahead Logging (WAL): Log record must be written to stable storage BEFORE the data page is written to disk. Ensures changes can be replayed or undone after crash.

Log record format: <TxnID, DataItem, OldValue, NewValue>

Strategy	Meaning
UNDO	Roll back uncommitted transactions (crash occurred before commit)
REDO	Re-apply committed transactions whose pages weren't flushed to disk
Deferred update	Write to log only; update DB only at commit → only REDO needed
Immediate update	Write to DB immediately; log old value → needs both UNDO + REDO

Checkpoint: Periodic sync of dirty pages to disk + write checkpoint record. Recovery only needs to process logs from last checkpoint → reduces recovery time.

Shadow Paging: Maintain two page tables (current + shadow). On commit, current becomes new shadow. Simple but high overhead for large DBs; rarely used today.

10. Backup Types

Type	What is backed up	Recovery
Full backup	Entire database	Restore full backup
Incremental	Changes since last backup	Full + all incrementals
Differential	Changes since last full	Full + latest differential
Point-in-time	Full + WAL logs	Recover to any point in time

Transactions & Concurrency Interview Checklist

- Explain ACID with real examples · Transaction states diagram · Serial vs serializable
- Draw precedence graph and detect cycles · Know all 4 concurrency problems
- Shared vs exclusive lock compatibility · 2PL growing/shrinking phases
- Strict 2PL advantage · Deadlock 4 conditions + detection/prevention
- All 4 isolation levels + which anomalies they prevent
- MVCC concept + advantage · WAL rule + why it matters · UNDO vs REDO
- Checkpoint purpose · Full vs incremental vs differential backup

Common Interview Traps: - Serializable isolation $\neq$ serial execution; it is equivalent. - MVCC still needs locking for writes in most implementations. - Dirty read involves uncommitted data; non-repeatable is about committed updates. - Phantom read involves new rows matching a query predicate, not existing row changes. - Strict 2PL prevents cascading rollbacks; basic 2PL does not.

UNIT 5 — STORAGE, INDEXING, QUERY PROCESSING & ADVANCED DBMS

1. Physical Storage & File Organisation

Storage hierarchy: CPU registers → Cache (L1/L2/L3) → RAM → SSD → HDD → Tape

Database → Tables → Pages/Blocks (8KB typical) → Records

File Type	How stored	Best for
Heap	Unordered, append	Bulk loads, full scans
Sequential	Sorted by key	Range scans on sort key
Hash	Hash function → bucket	Point lookups on hash key

Fixed-length records: Simple, easy to offset. **Variable-length:** More space-efficient; needs offset table. **Spanned:** Record splits across pages. **Unspanned:** Record fits in one page.

2. Indexing — Why

Without index: Full table scan — $O(N)$ rows read. **With index:** Navigate B+ tree → jump to relevant pages — $O(\log N)$.

Trade-off: Faster SELECT · Slower INSERT/UPDATE/DELETE (index must be maintained)
· Extra storage.

Why not index every column? Each index adds write overhead. n indexes → every INSERT updates n index structures.

3. Index Types

Index	Meaning
Primary	On PK; table physically ordered by it
Secondary	On non-PK column; separate structure
Clustered	Table data physically sorted by index key (only one per table)
Non-clustered	Separate index structure; pointer to row
Dense	Entry for every record
Sparse	Entry for every page/block (only clustered)
Composite	Index on (col1, col2, ...) — leftmost prefix rule

Covering	Index contains all columns needed by query (no table lookup)
Unique	Enforces uniqueness

Clustered	Non-Clustered
One per table	Many per table
Data rows sorted by key	Separate index + row pointer
Great for range queries	Great for point lookups

4. B-Tree vs B+ Tree ■

Feature	B-Tree	B+ Tree
Data stored	Internal + leaf nodes	Only leaf nodes
Leaf linking	No	Yes (linked list)
Range queries	Slower	Fast (traverse leaf list)
Space efficiency	Less (data in all nodes)	More compact internals
Used by DB engines	Rarely	Yes — InnoDB, PostgreSQL

Why B+ Tree for databases? Leaf-level linked list enables efficient range queries. Internal nodes store only keys → more keys per node → fewer levels → fewer disk reads.

B+ Tree operations: Search $O(\log n)$ · Insert $O(\log n)$ · Delete $O(\log n)$. Tree stays balanced always.

5. Hash Indexing vs B+ Tree

	B+ Tree	Hash Index
	Point O(log n) lookup	O(1) average
	Range Excellent query	Not supported
	Order Supported by	Not supported
	Best General purpose use	Equality lookups only

Collision handling: Chaining (overflow buckets). Dynamic hashing (extendible/linear) grows buckets as data grows.

6. Composite Index & Leftmost Prefix

```
CREATE INDEX idx ON users(city, age, name);
```

Usable for: WHERE city=... · WHERE city=... AND age=... · WHERE city=... AND age=... AND name=... NOT usable for: WHERE age=... alone (skips leftmost column).

7. Query Processing Pipeline

SQL Query

↓ Parser (syntax + semantic check, parse tree)

↓ Rewriter (view expansion, subquery rewrite)

↓ Optimizer (generates query plans, picks lowest cost)
↓ Executor (executes chosen plan)
↓ Storage Engine (reads pages, applies indexes)
↓ Result

Query Optimizer: Uses statistics (row counts, value distributions, histograms) to estimate cost of different plans. Chooses best join order, access method, and join algorithm.

8. Join Algorithms

Algorithm	Best when
Nested Loop Join	Small outer table or indexed inner
Block Nested Loop	Larger tables, uses buffer pool
Hash Join	Large unsorted tables (equi-join)
Sort-Merge Join	Both inputs already sorted or sort is needed

9. EXPLAIN / Execution Plan

```
EXPLAIN SELECT * FROM employees WHERE dept_id = 5;
```

Read the plan for: **access method** (Seq Scan vs Index Scan) · **join type** · **estimated rows** · **cost**.

- Seq Scan = full table scan → may need index.
- Index Scan = uses index → efficient for selective queries.
- Nested Loop / Hash Join / Merge Join → join strategies chosen by optimizer.

Interview Point: Always EXPLAIN before blaming a slow query on the database. The optimizer may not use an index if it estimates full scan is cheaper (low selectivity).

10. Distributed Databases

Horizontal scaling: Add more machines. **Vertical scaling:** Add more CPU/RAM to same machine.

Replication:	Sharding:
Primary → Replica 1	Shard 1 (users A–M)
→ Replica 2	Shard 2 (users N–Z)
Reads: any replica	Each shard holds partial data
Writes: primary only	

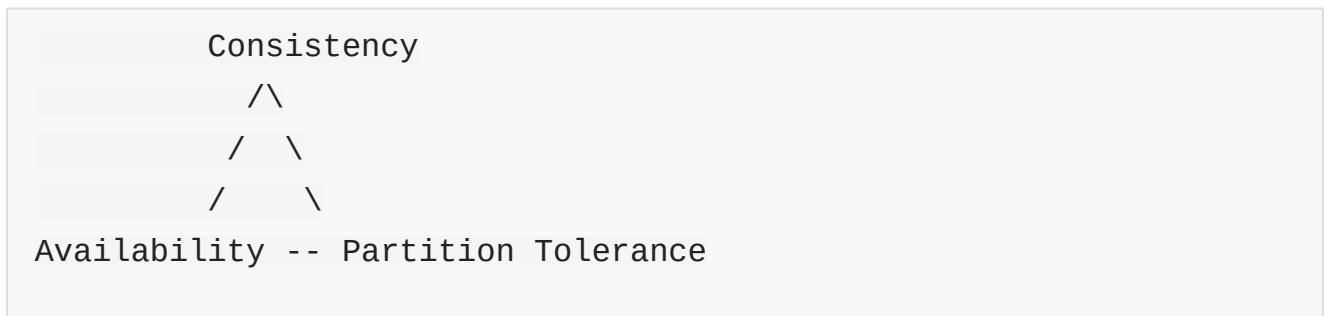
11. Replication

- **Purpose:** High availability, read scalability, disaster recovery.
- **Read replicas:** Distribute read load. Replication lag means replica may return stale data.
- **Failover:** Promote replica to primary on primary failure.
- **Interview:** Read replicas improve read throughput but do NOT guarantee strong consistency — there is always a replication lag window.

12. Sharding

- **Horizontal partitioning:** Each shard holds a subset of rows.
 - **Range-based:** Shard by value range (Jan–Jun → Shard1, Jul–Dec → Shard2). Risk: hot partitions if traffic skewed.
 - **Hash-based:** Hash(shard_key) % N → uniform distribution. Hard to range query.
 - **Consistent hashing:** Minimizes data movement when adding/removing shards.
 - **Shard key choice is critical:** Poor choice → hot partition → one shard overloaded.
-

13. CAP Theorem



In a distributed system: **Partition Tolerance is mandatory** (network failures happen). So the real trade-off: **CP** (consistent, may be unavailable during partition) vs **AP** (available, may return stale data).

- **CP example:** HBase, Zookeeper — returns error rather than stale data.
- **AP example:** Cassandra, DynamoDB — always responds, eventually consistent.

Interview Point: "Choose any two" is a simplification. Partition tolerance is not optional in distributed systems. The real choice is consistency vs availability during a partition.

14. NoSQL

Type	Example	Best for
Key-Value	Redis, DynamoDB	Sessions, caching, simple lookups
Document	MongoDB	Flexible schema, JSON data
Column-Family	Cassandra, HBase	Time-series, wide-row analytics
Graph	Neo4j	Relationships, social networks

Why NoSQL? Schema flexibility · Horizontal scaling · High write throughput · Specific data models (graphs, documents).

SQL vs NoSQL

	SQL	NoSQL
Schema	Fixed	Flexible
Jobs	Strong	Weak/none
Transactions	Full ACID	Limited (varies)
Scaling	Vertical (mainly)	Horizontal
Consistency	Strong	Eventual (typically)
Use case	Structured, relational	Scale, flexibility, varied models

Interview: NoSQL $\neq$ faster than SQL. Both have trade-offs. Choice depends on workload, consistency needs, and data model.

15. Distributed Transactions & 2PC

Two-Phase Commit:

Phase 1 (Prepare): Coordinator asks all participants to prepare and lock resources.
Phase 2 (Commit): If all say YES → coordinator sends COMMIT.
If any says NO → coordinator sends ROLLBACK.

Problem: If coordinator crashes after prepare and before commit → participants wait indefinitely (blocking protocol).

16. Caching

	Cache	Database
	Microseconds	Milliseconds
	Non-persistent	Yes
	Small	Large
	Eventually consistent	Strong

Strategies: - **Read-through:** Read from cache; miss → load from DB → store in cache. - **Write-through:** Write to cache and DB simultaneously. - **Write-back:** Write to cache; async write to DB later (risk: data loss on crash). - **Cache invalidation:** Hardest problem — when and how to evict stale data.

17. Production Concepts (Brief)

- **Connection pooling:** Reuse DB connections instead of creating new ones per request (PgBouncer, HikariCP).
- **Slow query log:** Identifies queries exceeding a threshold — first step in optimization.
- **Query profiling:** Use EXPLAIN ANALYZE to see actual vs estimated rows.
- **DB migrations:** Version-controlled schema changes (Flyway, Liquibase).
- **Data archival:** Move old data to cold storage; keeps active table small and fast.

Unit 5 Interview Checklist

- Physical storage hierarchy and page concept · Heap vs sequential vs hash files
- Why indexes exist and their write trade-offs · Clustered vs non-clustered
- B+ tree structure: leaf linking, internal keys only — WHY used in DBs
- Hash index: O(1) lookup, no range query support
- Composite index leftmost-prefix rule · Covering index concept
- Query processing stages and optimizer role · EXPLAIN output interpretation

- Join algorithms and when each is used
- Replication lag + failover · Sharding key importance + hot partitions
- CAP theorem: real trade-off is CP vs AP · Consistent hashing
- NoSQL types with examples · SQL vs NoSQL decision framework
- 2PC protocol and blocking problem · Cache strategies

Common Interview Traps: - Clustered index physically orders table data — only ONE per table. - Non-clustered indexes store a pointer back to the actual row. - B+ tree keeps all data at leaf level; B-tree stores data in all nodes. - Hash index cannot support range queries or ORDER BY. - NoSQL databases can have transactions (e.g., MongoDB multi-doc transactions). - CAP's "partition tolerance" is not optional in real distributed systems. - Replication improves read availability, not write scalability. - A covering index avoids a second lookup to the table ("index-only scan").

DBMS — LAST-MINUTE REVISION SHEET

Fundamentals

- DBMS: software managing data. RDBMS: uses relational model + SQL.
- 3-tier: client → app server → DB server.
- Schema = structure (rarely changes). Instance = current data (always changing).
- 3 abstraction levels: View → Logical → Physical.

SQL

- DDL/DML/DQL/DCL/TCL categories.
- DELETE (DML, rollbackable) ≠ TRUNCATE (DDL, faster) ≠ DROP (removes table).
- Logical order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
- WHERE ≠ HAVING: WHERE filters rows; HAVING filters groups.
- NULL ≠ 0 ≠ ". Use IS NULL. NULL comparisons yield UNKNOWN.
- Window functions: ROW_NUMBER (no ties), RANK (gaps), DENSE_RANK (no gaps).

Normalization

- 1NF: atomic values. 2NF: no partial deps. 3NF: no transitive deps. BCNF: every determinant is candidate key.
- 3NF → dependency preserving + lossless. BCNF → lossless but may lose dependencies.
- Denormalize for read-heavy/analytics workloads.

Transactions

- ACID: Atomic (all-or-nothing) · Consistent (valid states) · Isolated (appear serial) · Durable (survives crash).
- Transaction states: Active → Partially Committed → Committed | Failed → Aborted.

Concurrency

- Dirty read: read uncommitted data. Non-repeatable: same row changes between reads. Phantom: new rows appear.
- S-lock (read, shared OK). X-lock (write, exclusive). S+X = denied.
- 2PL: grow phase (acquire) → lock point → shrink phase (release). Strict 2PL: hold X-locks until commit.
- Deadlock: detect (wait-for graph), prevent (resource ordering), timeout.

Recovery

- WAL: write log before data page. Log: .
- UNDO uncommitted. REDO committed but not flushed.
- Checkpoint: reduces log replay on recovery.

Indexing

- B+ tree: keys in internals, data in leaves, leaves linked → range-friendly. $O(\log n)$.
- Hash index: $O(1)$ equality, no range queries.

- Clustered: table sorted by index key, one per table. Non-clustered: separate structure, many allowed.
- Covering index: all query columns in index → no table lookup.

Query Optimization

- Optimizer uses statistics + cardinality estimates to pick lowest-cost plan.
- EXPLAIN shows: access method, join type, estimated rows, cost.
- Seq Scan vs Index Scan choice depends on selectivity.

Distributed DB

- Replication: copies → read scalability, HA. Lag = eventual consistency risk.
- Sharding: horizontal split. Bad shard key → hot partitions.
- Consistent hashing: minimize data movement when adding shards.

NoSQL

- Key-Value (Redis), Document (MongoDB), Column (Cassandra), Graph (Neo4j).
- CAP: CP (consistent, may be unavailable) vs AP (available, eventually consistent). Partition tolerance is mandatory.
- SQL for structured + relational. NoSQL for scale + flexibility + specific models.

DBMS — INTERVIEW CHECKLIST

Fundamentals - [] What is DBMS? How does it differ from a file system? - [] DBMS vs RDBMS? What does "relational" mean? - [] Explain schema vs instance with an example. - [] What are the 3 levels of data abstraction? What is data independence? - [] Explain super key → candidate key → primary key chain. - [] Primary key vs unique key (NULL difference)? - [] What is a foreign key? What is referential integrity? ON DELETE CASCADE? - [] All ER attribute types. Strong vs weak entity. - [] How to convert M:N ER relationship to tables?

SQL - [] DDL vs DML vs DCL vs TCL — give examples. - [] DELETE vs TRUNCATE vs DROP — all differences. - [] SQL logical query processing order. - [] WHERE vs HAVING

— when to use each. - [] How does NULL behave in COUNT, AVG, comparisons? - [] Write INNER, LEFT, SELF JOIN from memory. - [] Write a correlated subquery. Explain how it runs. - [] UNION vs UNION ALL — difference and performance. - [] RANK vs DENSE_RANK vs ROW_NUMBER — give example with ties. - [] Write: 2nd highest salary, top-3 per department, employees > manager, delete duplicates. - [] What is a CTE? When to prefer over subquery? - [] What is a materialized view? How does it differ from a view?

Normalization - [] What is functional dependency? Give example. - [] Compute attribute closure step by step. - [] State 1NF, 2NF, 3NF, BCNF conditions precisely. - [] Identify partial and transitive dependencies in a given schema. - [] 3NF vs BCNF — what's the difference? Which preserves dependencies? - [] What is lossless decomposition? Why does it matter? - [] When would you denormalize? Give a real example. - [] Design schema for: e-commerce, college, banking, social media.

Transactions - [] Define transaction. Why do we need them? - [] Explain ACID with one example per property. - [] Draw transaction state diagram. - [] Serial vs serializable schedule — difference? - [] How do you check conflict serializability? (precedence graph) - [] What is a dirty read? Phantom read? Non-repeatable read?

Concurrency - [] Shared lock vs exclusive lock — compatibility? - [] Explain 2PL. What is the lock point? - [] Strict 2PL vs basic 2PL — what does strict add? - [] Deadlock 4 necessary conditions. How do you detect/prevent? - [] 4 isolation levels — which anomaly does each prevent? - [] What is MVCC? Why does it avoid read-write contention?

Recovery - [] What is WAL? Why must log be written before the data page? - [] UNDO vs REDO — when is each needed? - [] What does a checkpoint do? How does it reduce recovery time? - [] Full vs incremental vs differential backup — differences.

Indexing - [] Why do indexes exist? What is the trade-off? - [] B-tree vs B+ tree — key structural difference? - [] Why do databases use B+ trees for range queries? - [] Clustered vs non-clustered — how many per table? Data ordering? - [] B+ tree vs hash index — when to use each? - [] Composite index leftmost-prefix rule — give example. - [] What is a covering index? - [] Why not create an index on every column?

Query Processing - [] Query processing stages from SQL text to result. - [] What does the query optimizer do? What inputs does it use? - [] Read and interpret EXPLAIN output (Seq Scan, Index Scan, Hash Join). - [] When would the optimizer choose a Seq Scan over an Index Scan?

Distributed DB & NoSQL - [] Why replication? What is replication lag? Does it affect consistency? - [] Sharding vs replication — what each solves. - [] Shard key — why is the choice critical? What is a hot partition? - [] Consistent hashing — why is it better than simple

modulo hashing? - [] CAP theorem — what is the real trade-off? (CP vs AP, not "pick any 2") - [] 4 NoSQL types with examples and use cases. - [] SQL vs NoSQL — when would you choose each? - [] Two-Phase Commit — explain protocol + blocking problem. - [] Cache read-through vs write-through vs write-back. - [] What is connection pooling? Why is it needed?